CMSC 420: Amortized Analysis

Justin Wyss-Gallifent

January 15, 2026

1 Problems **2**

1 Problems

1. Aggregate method: Suppose we perform n operations where each operation costs 1.

   (a) What is the cost $C(n)$ of performing n operations?

   (b) What is the amortized per-operation cost of performing n operations?

2. Aggregate method: Suppose we perform n operations where the ith operation costs i.

   (a) What is the cost $C(n)$ of performing n operations?

   (b) What is the amortized per-operation cost of performing n operations?

3. Aggregate method: Suppose we perform n operations. Each operation costs 2 except every tenth operation costs an additional 14.

   (a) What is the cost $C(n)$ of performing n operations?

   (b) What is the amortized per-operation cost of performing n operations?

   (c) Prove that the amortized cost is $\mathcal{O}(1)$.

4. Aggregate method: Suppose we perform n operations. The ith operation costs $2i$ except every tenth operation costs an additional i (so the $i = 10$ operation costs an extra 10, the $i = 20$ operation costs an extra 20, and so on).

   (a) What is the cost $C(n)$ of performing n operations?

   (b) What is the amortized per-operation cost of performing n operations?

   (c) Prove that the amortized cost is $\mathcal{O}(n)$.

5. Token method: Suppose we execute a sequence of n operations on a data structure. Each operation has a base cost of 2 but every tenth operation has a supplemental cost of 4. Calculate the amortized cost. You can focus on a generalized run.

6. Token method: Suppose we are allocating memory for a stack. Each time a push would overflow the allocated memory we first reallocate to thrice the previous length, copy over the original stack, then do the push. The reallocation cost in tokens equals the new space allocated and this includes the copying of the old stack. Each push costs 1 token. Calculate the amortized cost. You can focus on a generalized run.

7. Token method: We insert n keys into an empty binary search tree at cost of 1 per insertion. When $n = 2^i$ for any $i \geq 0$ we rebuild the entire tree (it doesn't matter how) at a cost of 4^i. Prove that the amortized cost is $\mathcal{O}(n)$. You can focus on a generalized run.

8. Both methods: Consider amortized cost for the scenario where we have a stack such that:

   - A pop costs 1.
   - A push costs 1, not counting any allocation that may be necessary.
   - Re-allocation costs the new size. This cost includes copying the contents of the old stack.

   Assume we start with 0 bytes allocated and when we re-allocate we multiply the length by $\alpha \in \mathcal{Z}$ where $\alpha \geq 2$ except for the very first re-allocation which goes from 0 to 1.

   Define:

$$f(\alpha) = \frac{\alpha^2 + \alpha - 1}{\alpha - 1}$$

   (a) Use the aggregate method to prove that in the worst case with n operations the amortized cost $AC(n)$ of each stack operation satisfies $AC(n) \leq f(\alpha)$.

   (b) Prove mathematically that this expression is minimum when $\alpha = 2$.

   (c) Use the token method to prove that collecting $f(\alpha)$ tokens per push/pop is sufficient. As in class you can ignore the first and last runs.

9. Aggregate method: Consider amortized cost for the stack allocation problem where:

    - A pop costs 1.
    - A push costs 1, not counting any allocation that may be necessary.
    - When reallocation is necessary we multiply the length by 3.
    - Reallocation costs the size of the new list and includes copying the old elements over.
    - The list starts with one space allocated and nothing in it.

    Suppose that we do some sequence of n pushes and pops. Use the aggregate method to prove that the amortized cost is less than 5.5.

10. Aggregate method: Suppose you are working with a stack allocation onto which you will push n items.

    You start with zero entries allocated and decide that when a push will overflow the list you will first reallocate to the next highest perfect square, copy over the old stack entries, then push.

    For example the $i = 1$ append overflows and hence requires a reallocation to length 1. The $i = 2$ append overflows and hence requires a reallocation to length 4. The $i = 3, 4$ appends require no reallocation but the $i = 5$ append overflows and hence requires a reallocation to length 9. And so on.

    Assuming that each append cost 1 and reallocating costs the size of the new list, including the copying, prove that the amortized cost is $\mathcal{O}(\sqrt{n})$.

    Use the aggregate method.

11. Aggregate method: Suppose we have an list A which represents a binary string, so $A[0]$ represents the $2^0 = 1$s digit, $A[1]$ represents the $2^1 = 2$s digit, $A[2]$ represents the $2^2 = 4$s digit, and so on.

    Suppose A starts at all 0s and we then implement n increment operations. Every time a bit flips the cost is 2^k where 2^k is the index of the bit.

    Show that $AC(n) = \mathcal{O}(\lg n)$.

12. Token method: Revisiting the stack allocation problem suppose that when reallocation is necessary we multiply the length by some constant $C \in \mathcal{Z}$ with $C > 1$. Each push costs 1 and a reallocation cost equals the number of bytes being allocated and this includes copying the old data.

    (a) Use the token method to find an upper bound on the amortized cost. You can focus on a generalized run.

    (b) As $C \to \infty$, what happens to $AC(n)$?

13. Aggregate method: Consider amortized cost for the scenario where:

    - We start with one position allocated but empty so a first push will simply fill that position and not require reallocation.
    - A pop costs 1.
    - A push costs 1, not counting any allocation that may be necessary.
    - When reallocation is necessary we multiply the length by 5.
    - Reallocation costs the size of the new list and includes copying the old elements over.

    Suppose that we do some sequence of n pushes and pops. Use the aggregate method to prove that the (worst-case) amortized cost is less than 7.5.

14. Aggregate method: Suppose we are working on a stack onto which we will push n items.

    We start with zero entries allocated and decide that when a push will overflow the list we will reallocate to the next highest perfect cube length.

    For example the $i = 1$ push overflows and hence requires a reallocation to length 1. The $i = 2$ push overflows and hence requires a reallocation to length 8. The $i = 3, 4, 5, 6, 7, 8$ pushes require no reallocation but the $i = 9$ append overflows and hence requires a reallocation to length 27. And so on.

    Assuming that each push costs 1 and reallocating costs the size of the new list, including the copying, prove that the worst-case amortized cost is $\mathcal{O}\left(\sqrt[3]{n}\right)$.

15. Token method: Suppose we are working on a stack onto which we will push n items.

    We start with zero entries allocated and decide that when a push will overflow the list we will reallocate to the next highest perfect cube length. For example the $i = 1$ push overflows and hence requires a reallocation to length 1. The $i = 2$ push overflows and hence requires a reallocation to length 8. The $i = 3, 4, 5, 6, 7, 8$ pushes require no reallocation but the $i = 9$ append overflows and hence requires a reallocation to length 27. And so on.

    Assuming that each push costs 1 and reallocating costs the size of the new list, including the copying, prove that the worst-case amortized cost is $\mathcal{O}\left(\sqrt[3]{n}\right)$.

    You can focus on a generalized run.

16. Aggregate method: We insert n keys into an empty binary search tree at cost of 1 per insertion. When $n = 2^i$ for any $i \geq 0$ we rebuild the entire tree (it doesn't matter how) at a cost of 4^i.

    (a) Find an expression for the exact cost $C(n)$ of inserting the n keys.

    (b) It is a fact that $C(n) < n + 4^{\lfloor \lg n \rfloor + 1}$ (this is given in case you make a mistake in (a)). Use this fact to prove that $AC(n) = \mathcal{O}(n)$.

17. Suppose you are doing the aggregate method of amortized analysis and you find that:

$$C(n) = 8^{\lceil \lg n \rceil} - 1$$

Prove that $AC(n) = \mathcal{O}(n^k)$ for an appropriate integer k.

Solution:

18. Suppose we insert n keys into a binary search tree which starts empty. Each insertion costs 7 but whenever $n = 2^i - 1$ for each $i \geq 1$ we rebuild the binary tree (it doesn't matter how) at a cost of n; so we rebuild at each of $n = 1, 3, 7, 15, ...$ at a cost of $1, 3, 7, 15, ...$ respectively. Calculate the total exact cost $C(n)$ for performing these n insertions. Show all work and justify (at least a few words!) where any floors and/or ceilings come from. You should evaluate any resulting sums but you don't need to simplify or bound the result or find $AC(n)$.

    Solution:

19. Suppose we insert n keys into a binary search tree which starts empty. Each insertion costs 1 but whenever $n = 2^i - 1$ for each $i \geq 1$ we rebuild the binary tree (it doesn't matter how) at a cost of n; so we rebuild at each of $n = 1, 3, 7, 15, ...$ at a cost of $1, 3, 7, 15, ...$ respectively.

Using the token method we say a run starts after a rebuild has just occurred and ends after the next rebuild so for each $k \geq 0$ a run goes from $2^k - 1$ to $2^{k+1} - 1$ keys.

(a) If we deposit β tokens per operation, how many tokens will we deposit during a general (k is unknown) run?

Deposit	

(b) What will the cost of a general (k is unknown) run be? Simplify.

Cost	

(c) What it the smallest β such that we will deposit enough to cover the costs? Show your work.

Solution: